

Synthetic Deliveries

David Ferreira Quaresma (david.quaresma@ccc.ufcg.edu.br)

set, 2025

Introduction

This document evaluates simulations of hedged requests in FaaS context. The scenarios of study Analyzed here are baseline (pure faas environment without any hedged request technique) vs simplehr (simple hedged requests, meaning all function invocations are duplicated and considered the one with lower latency). PS: current simulator model uses the highest value on the trace’s input as coldstart. So we expect to have as much additional values near p100 as the amount of replicas created in simulation time.

The selected functions latencies to be studied here are described using discrete functions synthetized, such as a variation of bernoulli (function with two possible latency values, 1 or 10, 10 occurring 5% of the times), lognormal (tailered distribution defined with sigma and mu both as 1) and weibull (tailered distribution defined with k as 0.5, and lambda as 5.0). With bernoulli we want to understand the simplest scenario, with lognormal and weibull we want to explore tailed distributions. The interarrival for all 3 kinds of function is the same, constante, requests arriving every 5ms.

Results evaluation of Simple Latency Model Simulator

Simple Latency Model Simulator is a simplified version that does not considers coldstart or replica management impact. It only generated latencies for baseline and simplehr. It generates one latency value for baseline and two for simplehr, we compare the lower latency generated for simplehr with the one for baseline aiming for latency improvement evaluation, and compare the sum of the generated latencies for simplehr vs the one generetated for baseline to estimate the impact.

The Latency Improvement Comparison table shows both the improvements for latency and the total latency processed by each technique in the system to achieve the latency the user would perceive. Note that while latency were improved from ~30% up to 75%, the “cost” of it (meaning the total latency processed to achieve this goal), the total latency was almost twice for all function with simplehr technique. Improvement column is always simplehr in comparison for baseline, thus positive values means simplehr is better, negative otherwise.

Table 1: Simple Model Latency Improvement Comparison

scenario	baseline	simplehr	improvement
func-bernoulli-latency	217257.0	153447.0	29.37
func-bernoulli-totalLatency	217257.0	436269.0	-100.81
func-logNormal-latency	671411.4	321940.9	52.05
func-logNormal-totalLatency	671411.4	1345183.6	-100.35
func-weibull-latency	1496216.2	374258.4	74.99
func-weibull-totalLatency	1496216.2	2987437.3	-99.67

Table 2: Latency Improvement Comparison

scenario	baseline	simplehr	improvement
func-bernoulli-idletime-INF	218283.0	153474.0	29.69
func-bernoulli-idletime-15min	218283.0	153852.0	29.52
func-bernoulli-idletime-zero	1500000.0	1500000.0	0.00
func-logNormal-idletime-INF	686012.8	336013.9	51.02
func-logNormal-idletime-15min	774378.7	345737.2	55.35
func-logNormal-idletime-zero	38757514.9	38757514.9	0.00
func-weibull-idletime-INF	1587652.8	472711.1	70.23
func-weibull-idletime-15min	2436263.1	609675.6	74.97
func-weibull-idletime-zero	106402945.4	106402945.4	0.00

TLDR using tables for results using faas-simulator

Comparing baseline vs simplehr we attained the expect result of having lower tail latency distributions for simplehr at the cost of high function replica uptime (this is reasonable because that technique duplicate every function invocation).

The total latency sum improvement with simplehr was huge. Bernoulli total latency reduction was near ~30%, lognormal ~55%, and weibull ~75%. As long tailored the function latency distribution is, it seems the greater is the impact on the total latency distribution. As shown in later histogram at Latency Evaluations, this impact comes from reducing tail latency values.

Total up time for all functions was twice for zero idletime, and almost twice for inf idletime. However, for 15min idletime, up time was 1.7, 1.66, and 1.56 times higher for bernoulli, lognormal and weibull respectively. This suggests that depending on the idletime chosen for replicas, simplehr is able to not double up the up time due to its efficiency improvement.

Total Latency Improvement Comparison Table

This table summarizes the overall latency reduction (sum of response times) achieved by simpleHR compared to baseline under different idletime scenarios. The purpose is to quantify the percentage of performance improvement for each simulated function.

- baseline: total time in which a function replica was processing an invocation.
- simplehr: total time in which a function replica was available idle or processing an invocation.
- Improvement: ratio baseline latency / simplehr latency (%).

As expected since both scenarios are 100% impacted by coldstart on scenario idletime zero, we see zero improvements. However, for idletime inf and 15min we see huge latency improvements pro simplehr from ~29.5% up to ~75% depending on the latency distribution evaluated.

Table 3: Replicas’s Efficiency Usage

scenario	Total Busy Time		Total Up Time		Efficiency	
	baseline_b	simplehr_b	baseline_u	simplehr_u	baseline_e	simplehr_e
func-bernoulli-idletime-INF	218283.0	437313	1499986	2999977	14.55	14.58
func-bernoulli-idletime-15min	218283.0	439239	1499986	2551112	14.55	17.22
func-bernoulli-idletime-zero	1500000.0	3000000	1500000	3000000	100.00	100.00
func-logNormal-idletime-INF	686012.8	1376327	38993113	77986224	1.76	1.76
func-logNormal-idletime-15min	774378.7	1504065	2744901	4574789	28.21	32.88
func-logNormal-idletime-zero	38757514.9	77515030	38757515	77515030	100.00	100.00
func-weibull-idletime-INF	1587652.8	3175620	106449322	212898557	1.49	1.49
func-weibull-idletime-15min	2436263.1	4186868	5880390	9226709	41.43	45.38
func-weibull-idletime-zero	106402945.4	212805891	106402945	212805181	100.00	100.00

Table totals for uptime, busytime and efficiency

This section analyzes how hedged requests affect replica management. The purpose is to evaluate how baseline and simpleHR differ in terms of resource utilization efficiency. The table summarizes replica usage for each function and scenario:

- Total Busy Time: total time in which a function replica was processing an invocation.
- Total Up Time: total time in which a function replica was available idle or processing an invocation.
- Efficiency: ratio busy/up (%).

The expected result for idletime zero was attained, which is zero improvement as both technique scenarios are greatly impacted by coldstarts. Interestingly, for idletime inf we see equal performance. Lastly, idletime 15min simplehr has a slight advance, meaning it not only improves latency overall but does not penalty resource usage at all.

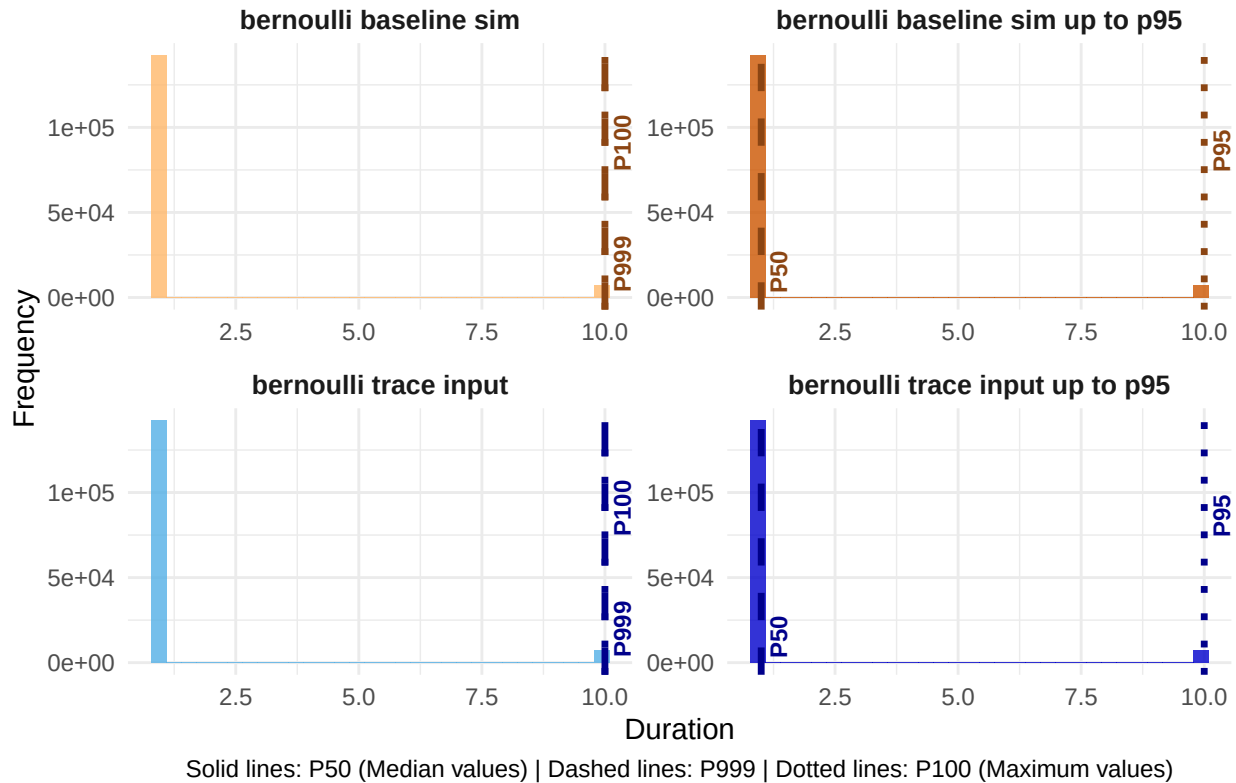
Latency Evaluations

In this section we will perform a series for latency evaluations. Each comparison will be using 4 histograms for latency, one considering the whole distribution, the second up to the percentile 95, then starting after p95 and lastly starting after p99. The main goal here is to understand the tehcnique impact on latency distribution in different portions of it.

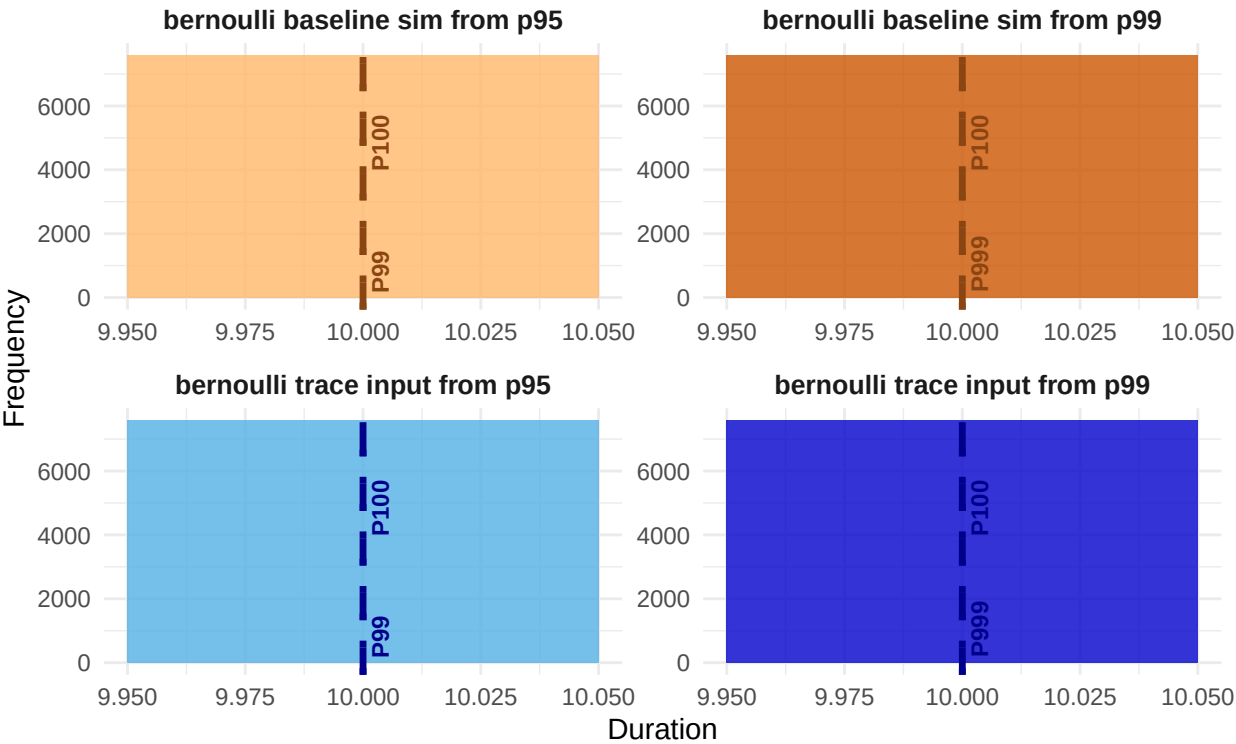
Sanity Tests comparing generated traces (Bernoulli, LogNormal & Weibull) to simulated baseline at idletime INF

These histograms compare the synthetic traces with the simulated baseline response times (with infinite idletime). The purpose is to validate the fidelity of the simulator against the trace input. We use infinite idletime here to reduce coldstart influence at maximum. The expected difference here observed is that for each coldstart simulated, the baseline simulated tail latency go towards p100. As shown in the histograms, the overall latency distribution is similar, but the expected coldstart present in simulation make the distribution divergence visually for lognormal and weibull on histogram plots starting after p99.

Sanity Check for bernoulli: Generated vs Simulated

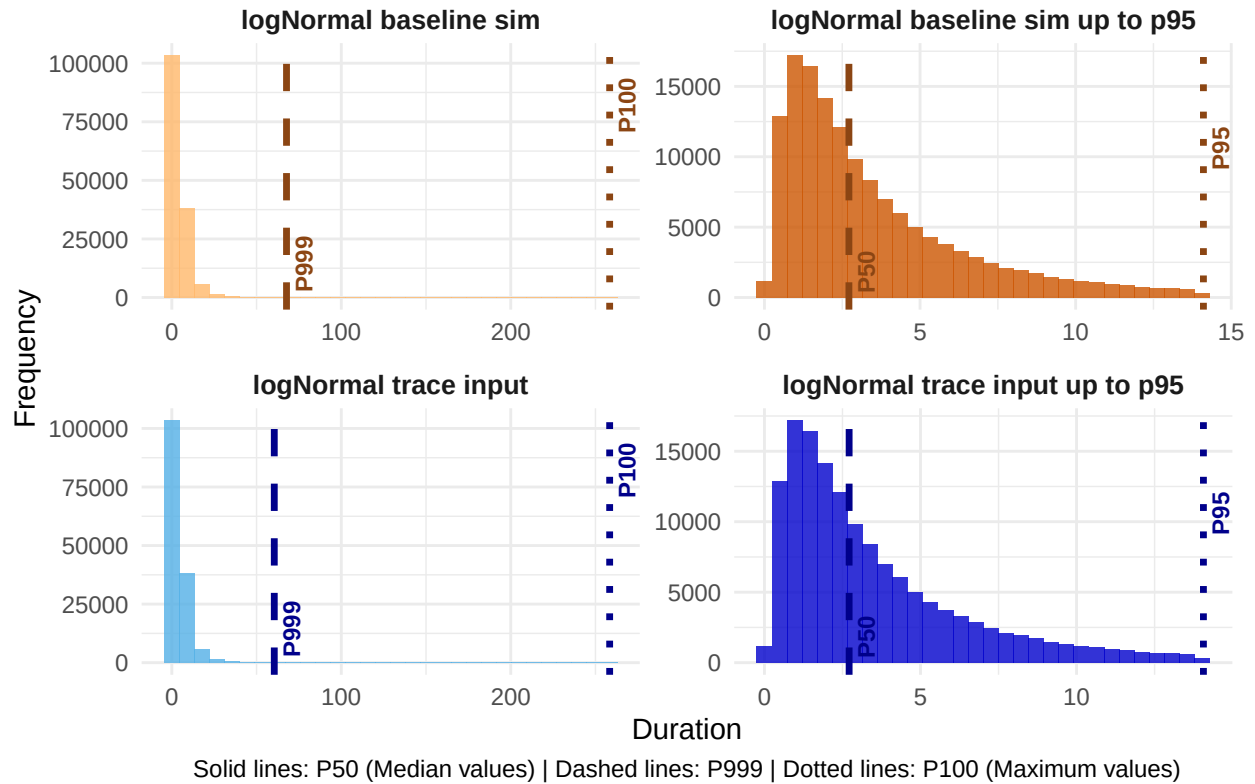


Sanity Check for bernoulli: Generated vs Simulated

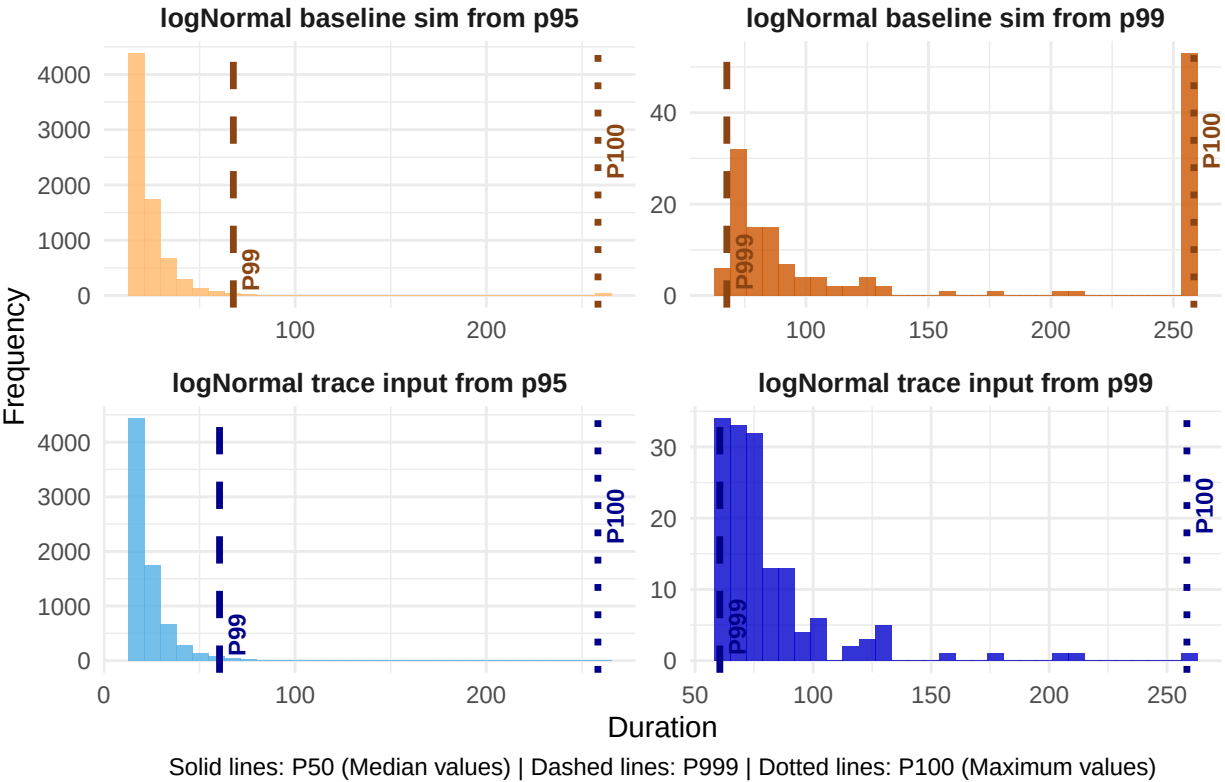


Solid lines: P50 (Median values) | Dashed lines: P999 | Dotted lines: P100 (Maximum values)

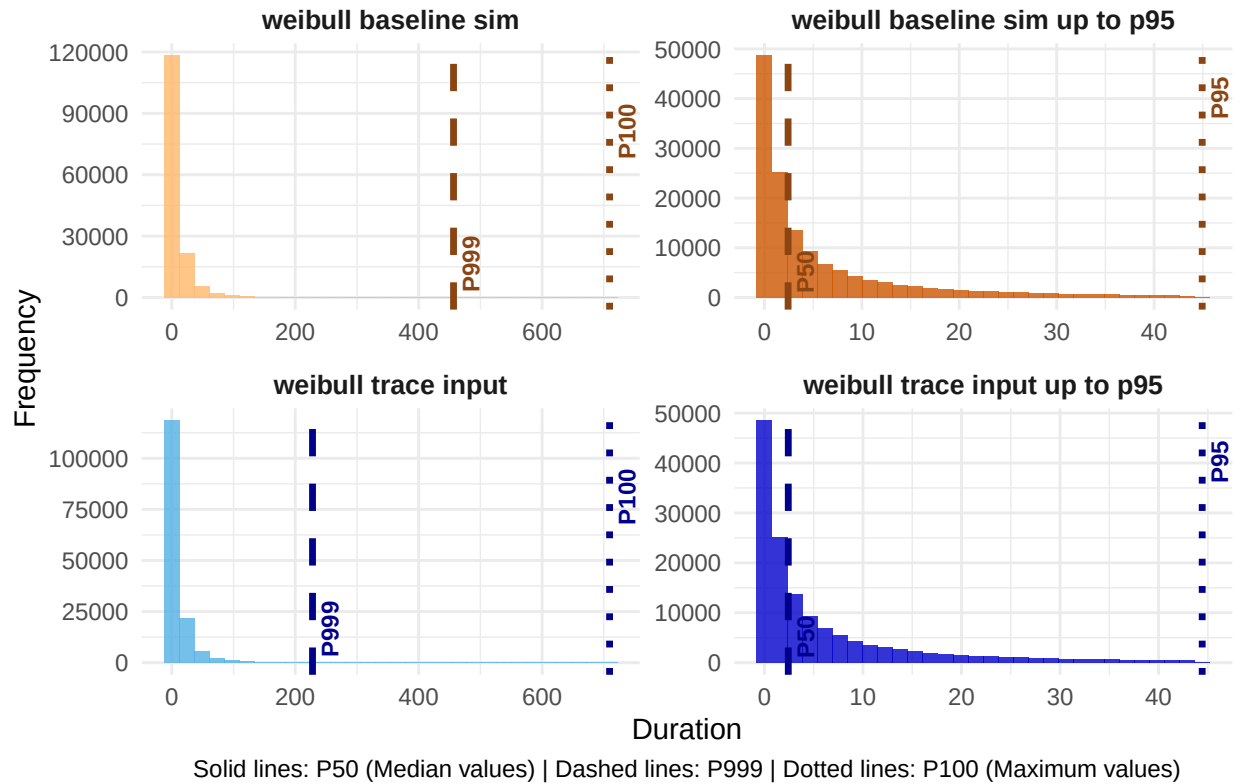
Sanity Check for logNormal: Generated vs Simulated



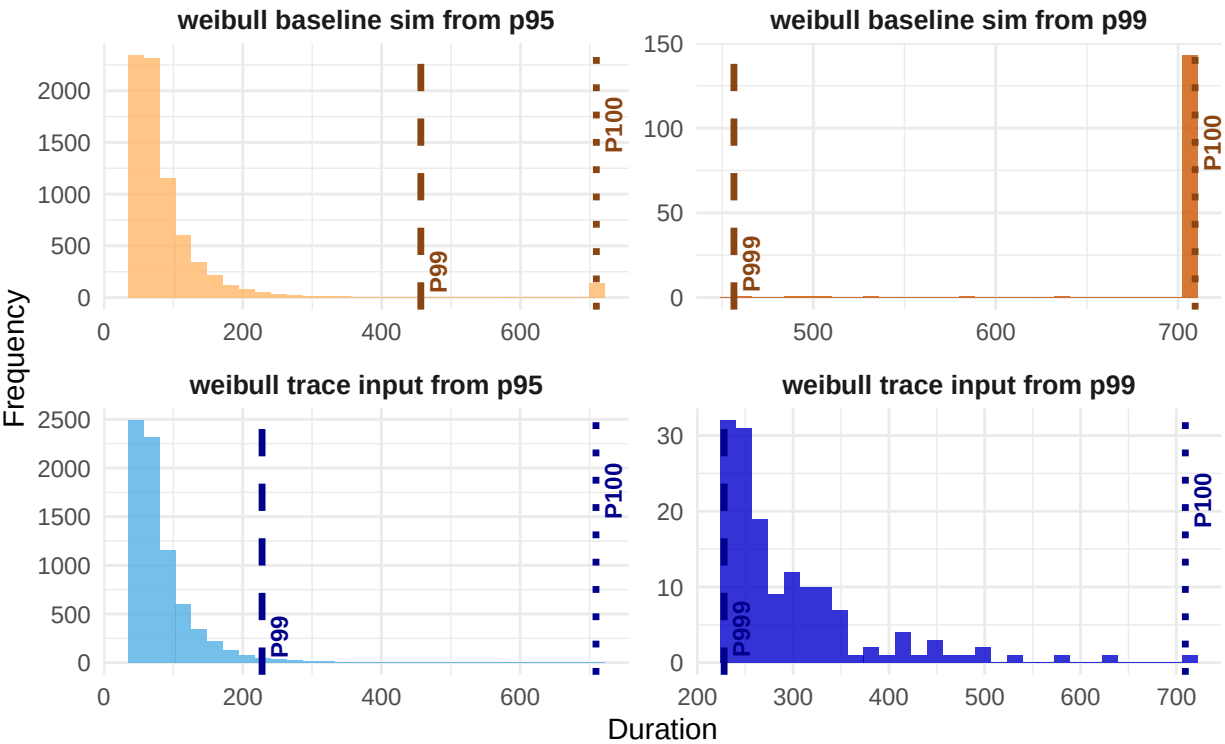
Sanity Check for logNormal: Generated vs Simulated



Sanity Check for weibull: Generated vs Simulated



Sanity Check for weibull: Generated vs Simulated

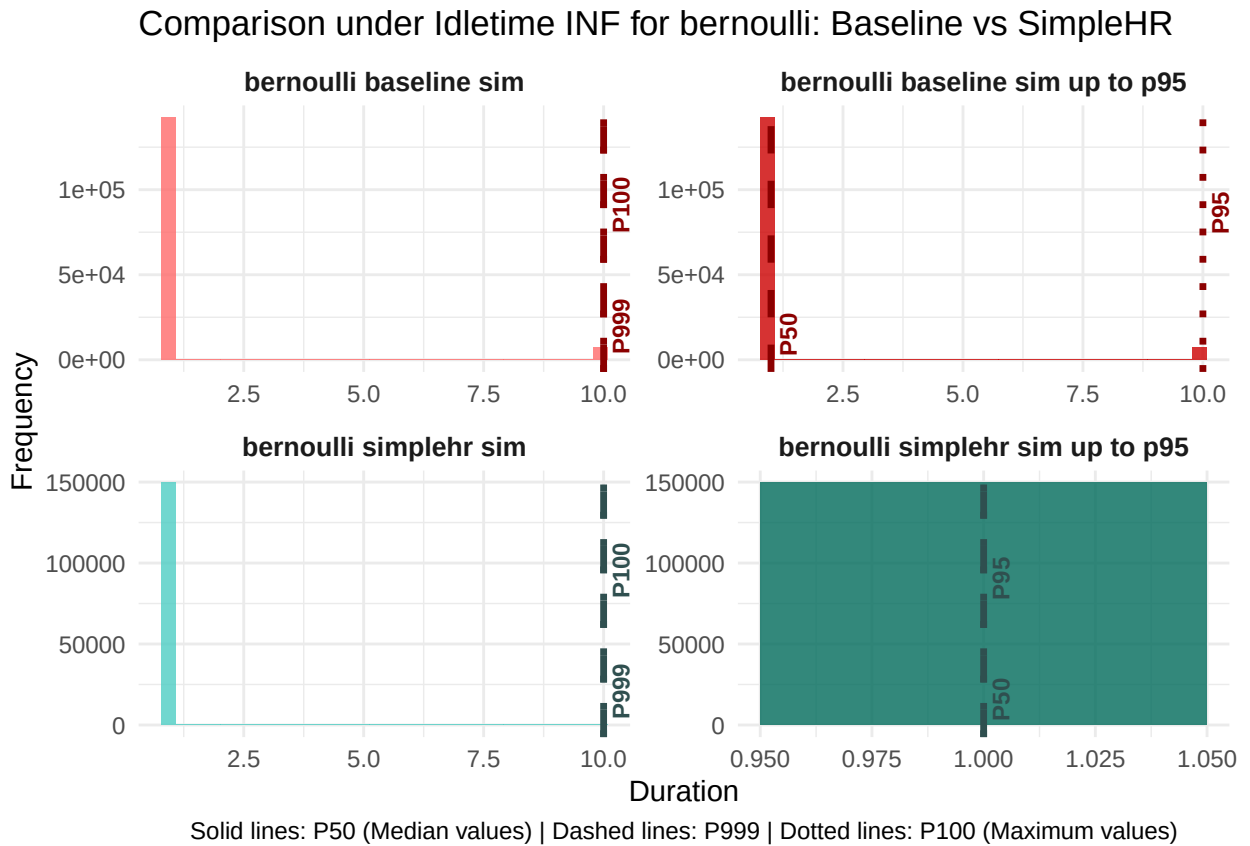


Solid lines: P50 (Median values) | Dashed lines: P99 | Dotted lines: P100 (Maximum values)

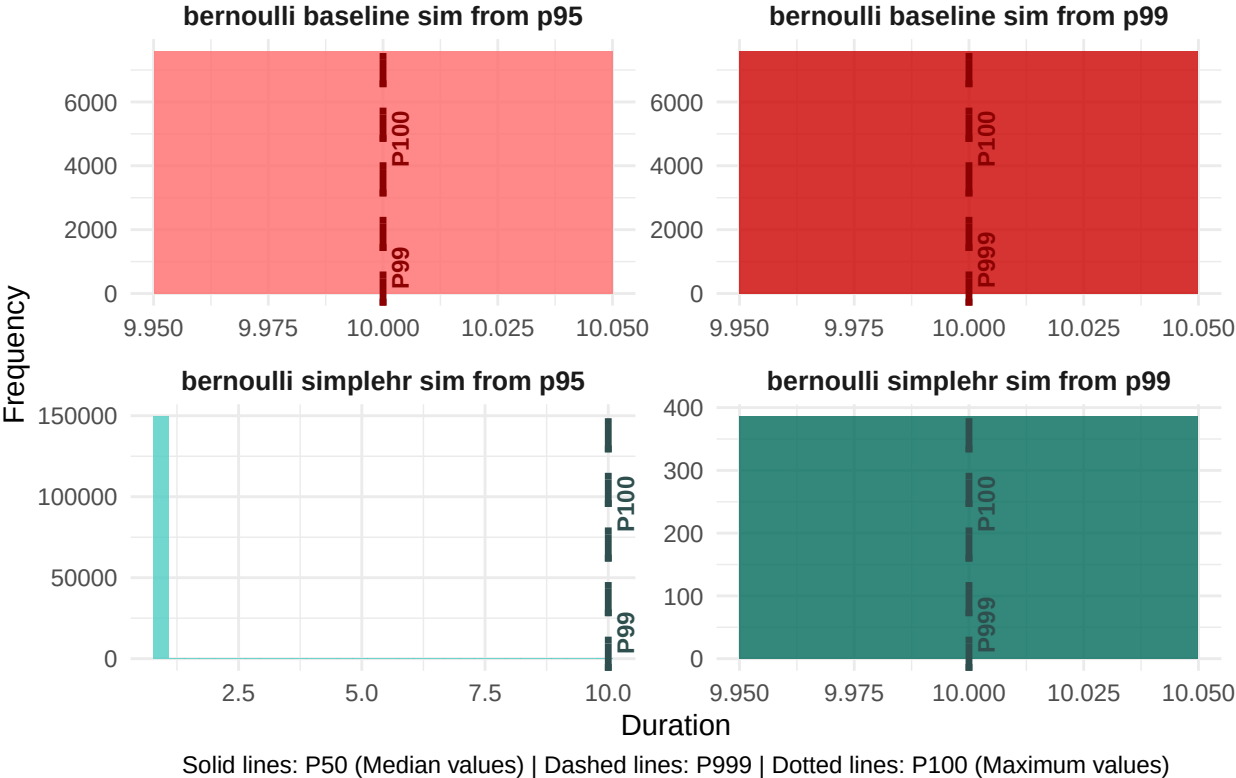
Simulated: SimpleHR vs Baseline with idletime as INF

This comparison contrasts baseline vs simpleHR (with infinite idletime). The purpose is to analyze how hedged requests affect latency in scenarios where functions are never deallocated. In the following bullets I'll summarize each results observed:

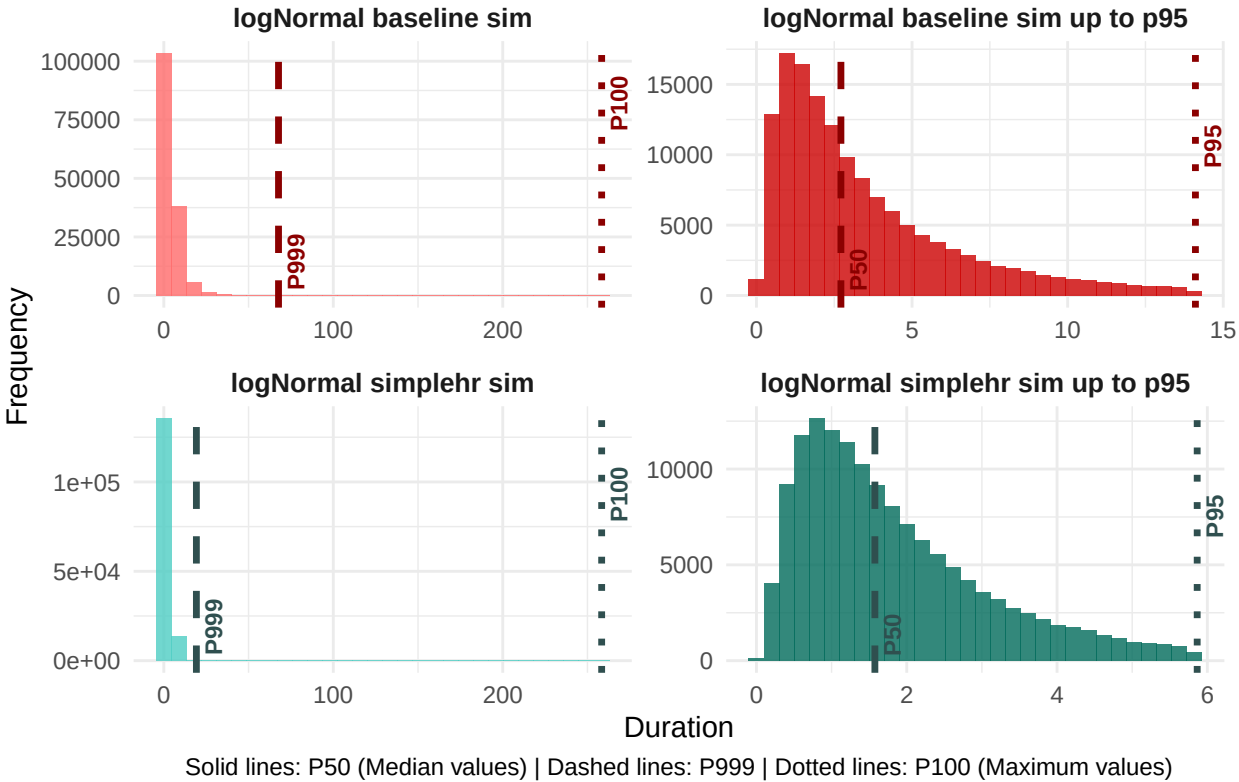
- Bernoulli: as we can see in histograms for up to p95, the simplehr (green) has all of its values around 1ms while baseline has p95 near 10ms. Thus, we successfully reduced the tail latency for this scenario
- LogNormal: using histogram for the whole distribution and histogram starting after p95, we can clearly see that p99 and p999 for simplehr (green) are considerably lower. Other than that, we can't visualize further improvements, although that is somehow an indication that tail latency is also reduced using simplehr on a function with lognormal latency distribution.
- Weibull results are similar to LogNormal, but in different magnitude.



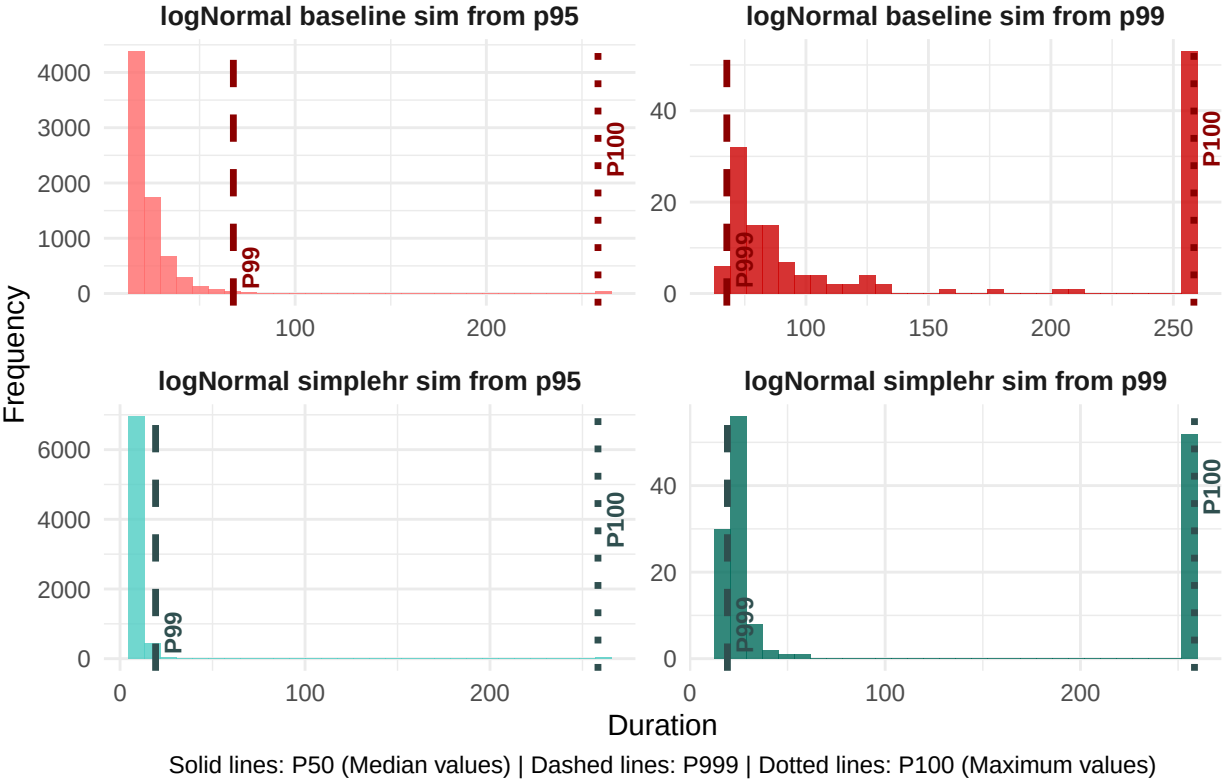
Comparison under Idletime INF for bernoulli: Baseline vs SimpleHR



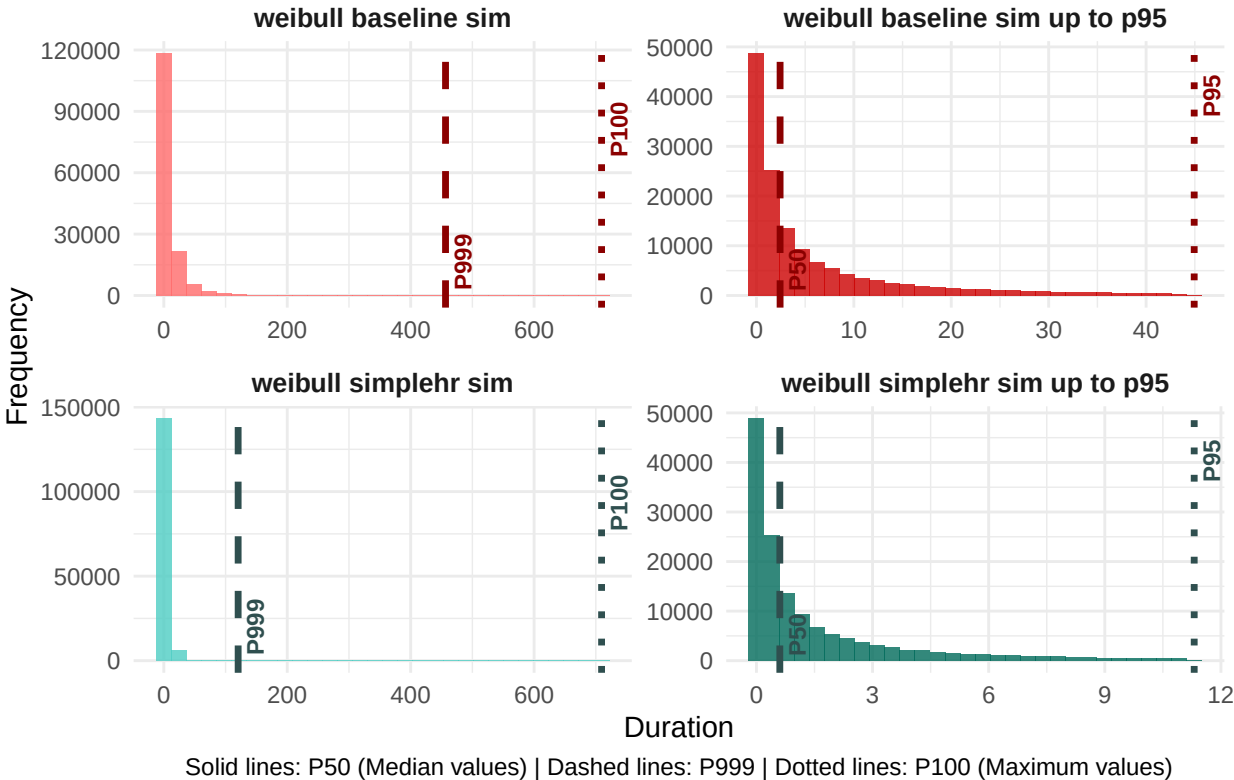
Comparison under Idletime INF for logNormal: Baseline vs SimpleHR



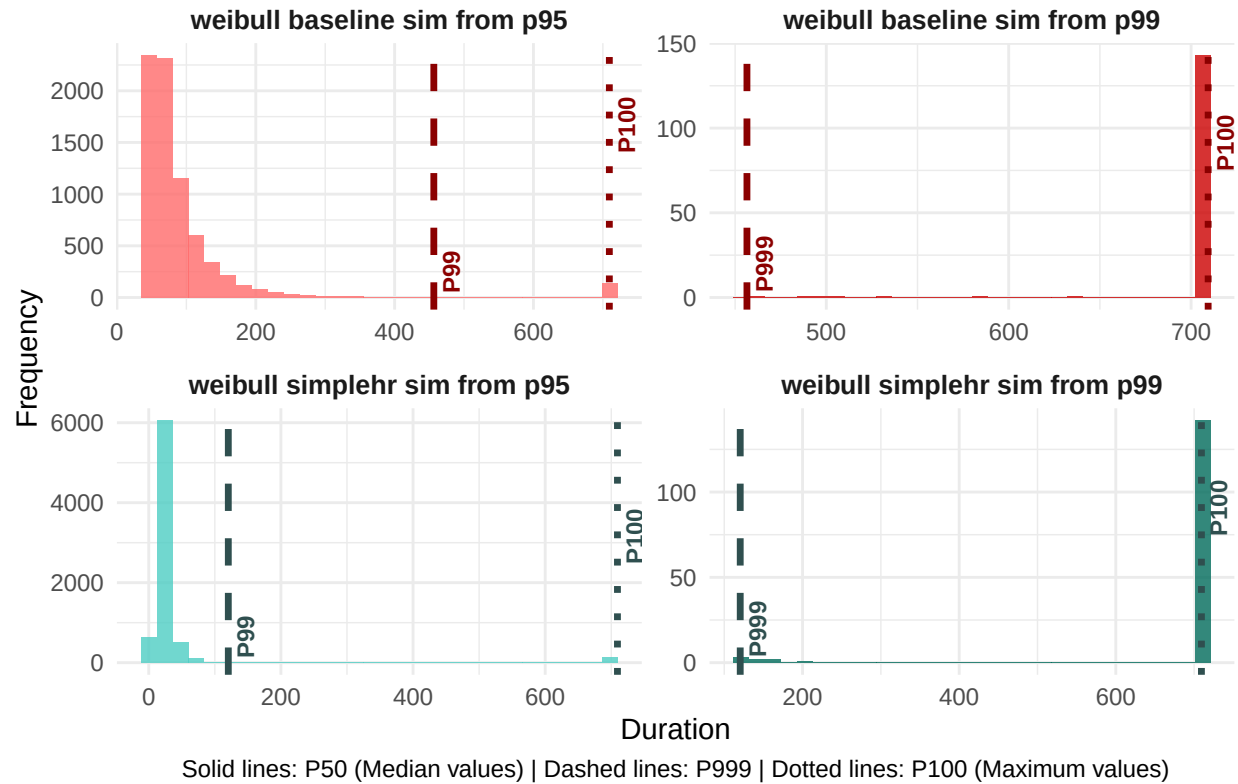
Comparison under Idletime INF for logNormal: Baseline vs SimpleHR



Comparison under Idletime INF for weibull: Baseline vs SimpleHR



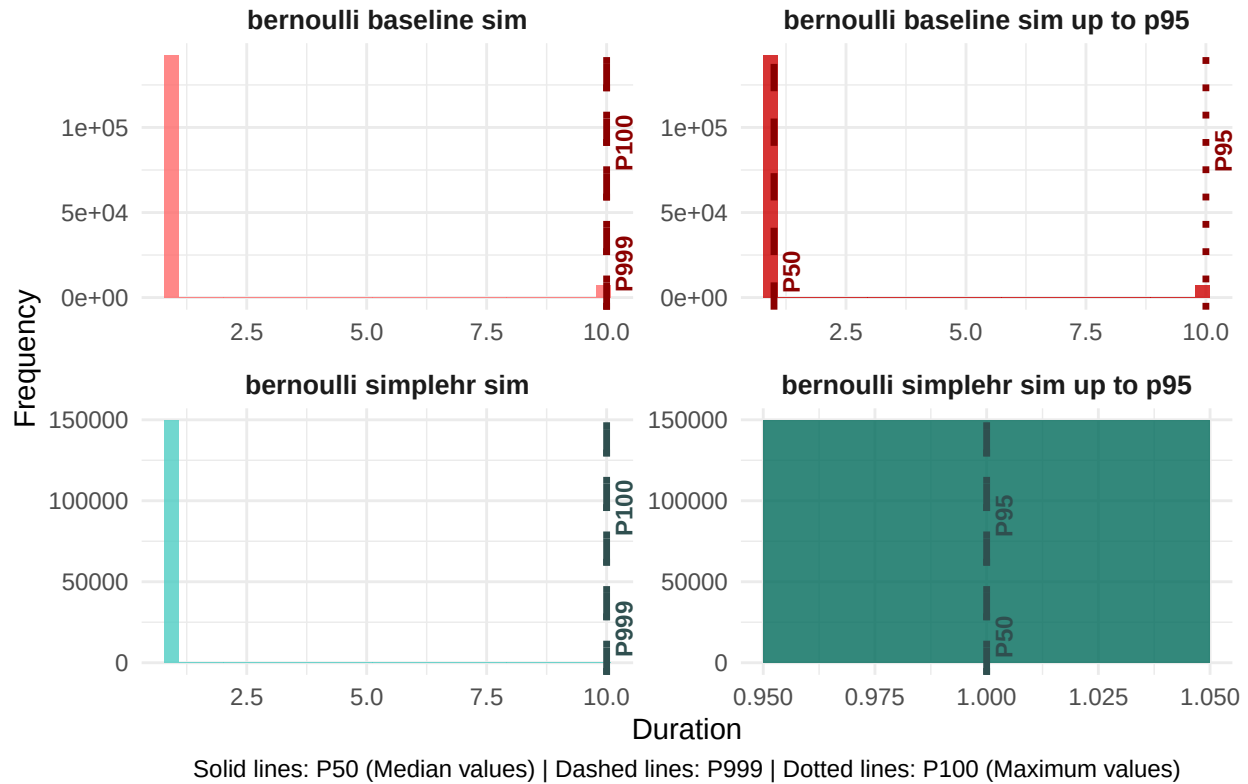
Comparison under Idletime INF for weibull: Baseline vs SimpleHR



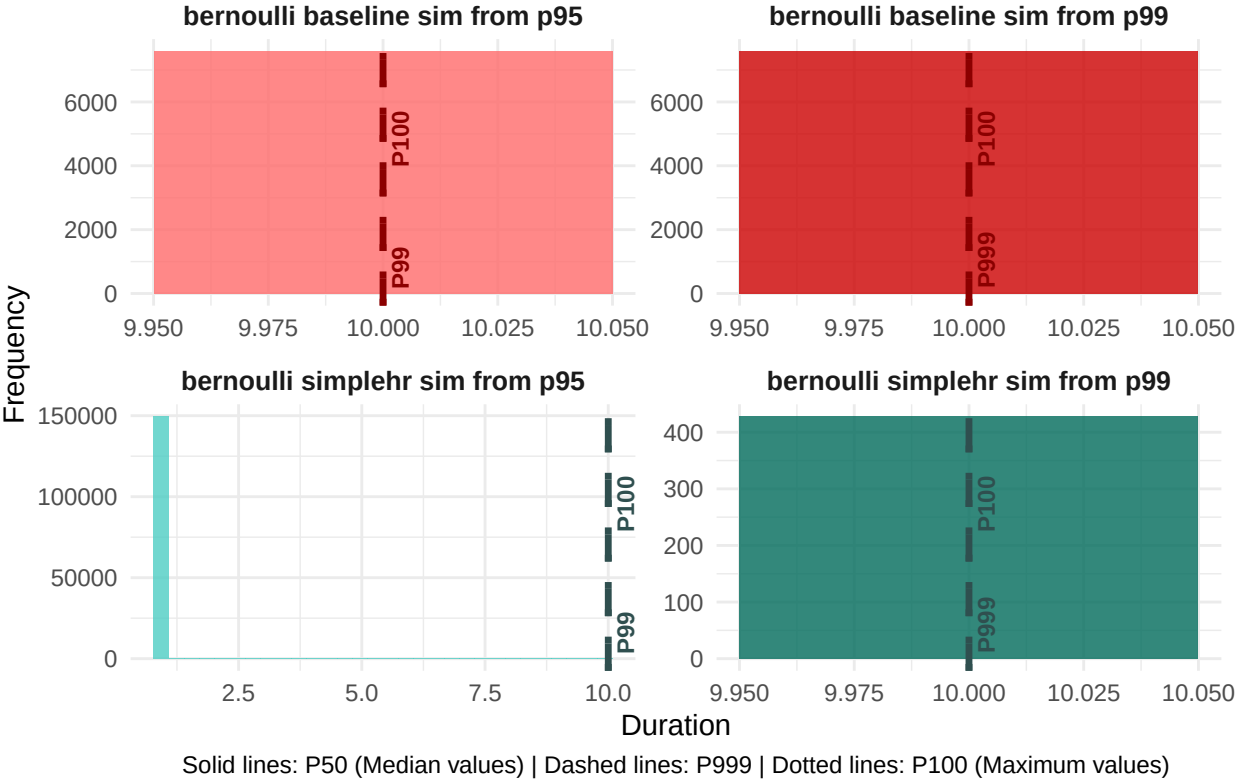
Simulated: SimpleHR vs Baseline with idletime as 15min

In this case, the idletime was set to 15 minutes. The purpose is to evaluate how replica deallocation policies influence the relative performance of simpleHR compared to baseline. For baseline and lognormal results were similar to using a idletime as inf, but for Weibull, we can't see differences on percentiles so to speak.

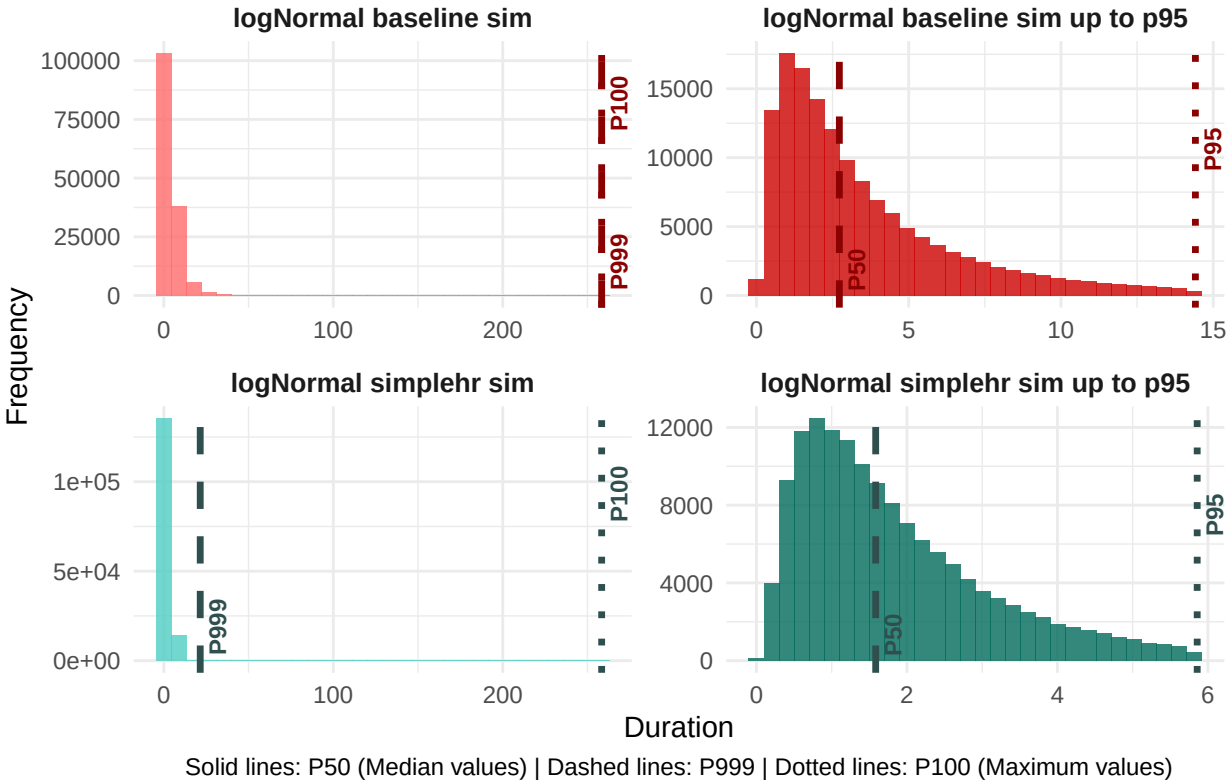
Comparison under Idletime 15min for bernoulli: Baseline vs SimpleHR



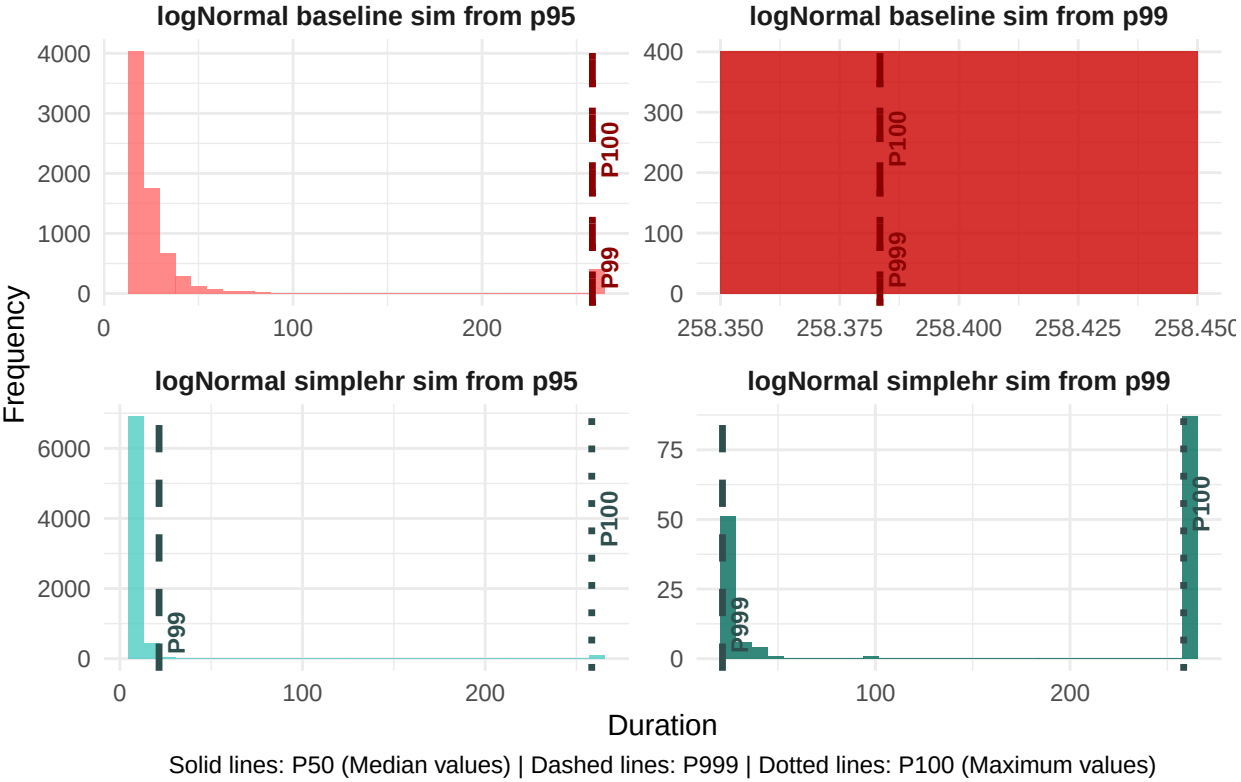
Comparison under Idletime 15min for bernoulli: Baseline vs SimpleHR



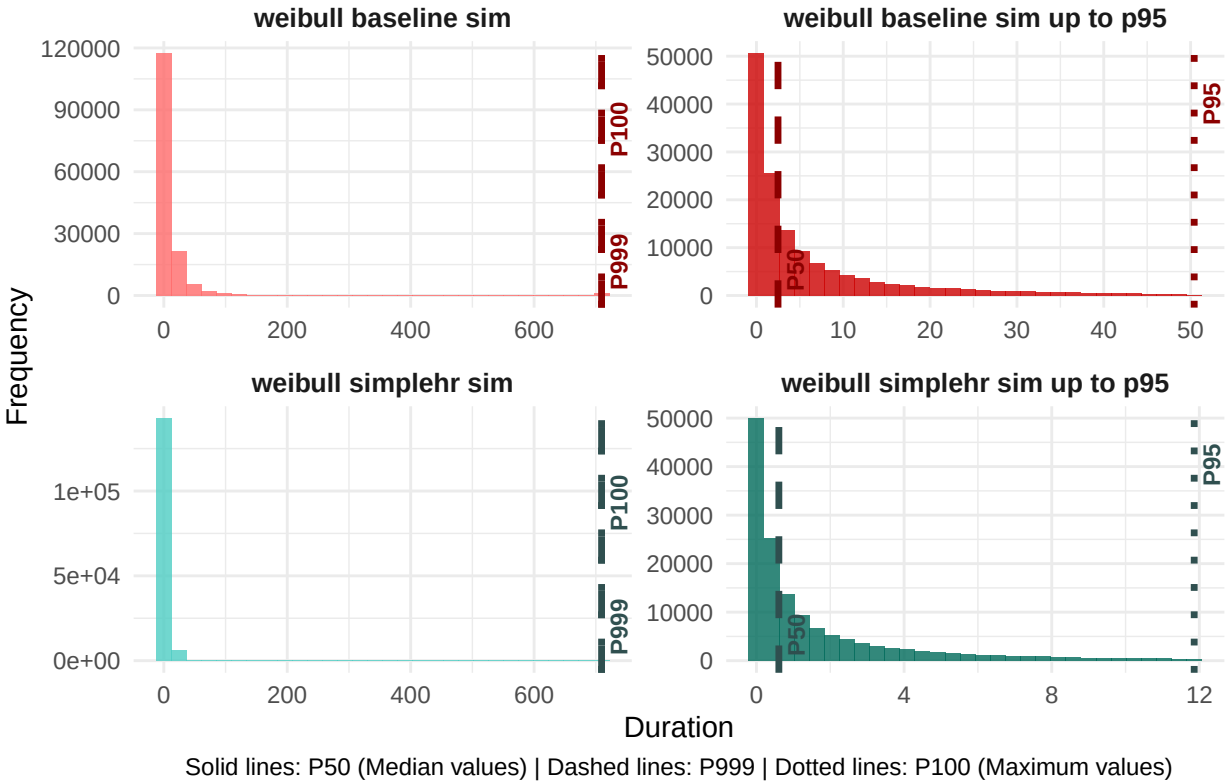
Comparison under Idletime 15min for logNormal: Baseline vs SimpleHR



Comparison under Idletime 15min for logNormal: Baseline vs SimpleHR



Comparison under Idletime 15min for weibull: Baseline vs SimpleHR



Comparison under Idletime 15min for weibull: Baseline vs SimpleHR

